



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
Disciplina: ALGORITMO E ESTRUTURA DE DADOS I

Semestre: 2026.2

Data: ____/____/____
PROFESSORA: Rosana Rego

— Exercício 8: Unidade I —

Módulos em C: separando entidade.h e entidade.c

Nome: _____

Nota: _____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade8.c`, evolução da `atividade7.c`, cujo objetivo é demonstrar, de forma visual e interativa (usando a biblioteca **raylib**), os seguintes conceitos de linguagem C:

- Separação de um programa em **múltiplos arquivos**: um cabeçalho (`entidade.h`) e uma implementação (`entidade.c`);
- **Compilação separada**: cada arquivo `.c` é compilado para um `.o` de forma independente;
- **Linkagem**: o *linker* junta os `.o`'s e resolve as chamadas entre eles, gerando um único executável;
- Reuso de um **módulo** (`entidade.h/entidade.c`) por diferentes programas, sem duplicar código;
- Revisão: ponteiros, alocação dinâmica, *struct*, **enum**, **union** e vetor de ponteiros para *struct*.

O jogo em si é o mesmo da atividade 5, mas agora o tipo `Entidade` e suas operações básicas moram em um módulo próprio, reutilizável por qualquer outro programa.

1 O cabeçalho `entidade.h` (interface)

O arquivo de cabeçalho declara os tipos (**enum**, **union**, *struct*) e os **protótipos** das funções do módulo. Qualquer arquivo `.c` que precisar de uma `Entidade` só precisa de `#include "entidade.h"`:

```
1 typedef enum {
2     ENTIDADE_JOGADOR,
3     ENTIDADE_INIMIGO,
4     ENTIDADE_ITEM
5 } TipoEntidade;
6
7 typedef union {
8     int dano;
9     int valor;
10 } ExtraEntidade;
11
12 typedef struct {
```

```

13     TipoEntidade   tipo;
14     Vector2        pos;
15     float          raio;
16     int            vida;
17     Color          cor;
18     ExtraEntidade  extra;
19 } Entidade;
20
21 Entidade *entidadeCriar(TipoEntidade tipo, Vector2 pos);
22 bool     entidadeColidiu(Entidade *a, Entidade *b);
23 void     entidadeDesenhar(Entidade *e);
24 void     entidadeAplicarDano(Entidade *e, int dano);

```

Note que a *struct* `Entidade` continua **visível** para quem inclui o cabeçalho – isso é um módulo comum, e não ainda um TAD (isso será visto na atividade 9).

2 A implementação entidade.c

O arquivo `entidade.c` implementa cada função declarada no cabeçalho. Ele é compilado separadamente e depois **linkado** ao programa principal:

```

1  Entidade *entidadeCriar(TipoEntidade tipo, Vector2 pos) {
2      Entidade *e = (Entidade *)malloc(sizeof(Entidade));
3      if (e == NULL) return NULL;
4
5      e->tipo = tipo;
6      e->pos  = pos;
7      /* ... preenchimento dos demais campos, igual a criarEntidade da
8         atividade5 ... */
9      return e;
10 }
11
12 void entidadeAplicarDano(Entidade *e, int dano) {
13     if (e == NULL) return;
14
15     e->vida -= dano;
16     if (e->vida < 0) e->vida = 0;
17 }

```

3 O programa principal usando o módulo

O arquivo `atividade8.c` inclui `entidade.h` e usa apenas a **interface pública** do módulo – ele não precisa saber *como* `entidadeCriar` ou `entidadeColidiu` são implementadas, apenas *o que* elas fazem:

```

1  #include "entidade.h"
2
3  Entidade *jogador = entidadeCriar(ENTIDADE_JOGADOR, posInicial);
4  adicionarEntidade(jogador);
5
6  /* ... */
7
8  if (entidadeColidiu(jogador, e)) {
9      entidadeAplicarDano(jogador, e->extra.dano);
10 }

```

A coleção de entidades (`vetorEntidades`, `adicionarEntidade`, `removerEntidade`) continua em `atividade8.c`, pois é uma regra específica **deste jogo**, e não algo genérico o bastante para entrar no módulo `entidade`.

Compilando múltiplos arquivos

Arquivo	Papel
<code>entidade.h</code>	declarações (interface do módulo)
<code>entidade.c</code>	implementação do módulo
<code>atividade8.c</code>	programa principal (usa a interface)

```
1 gcc atividade8.c entidade.c -o atividade8 -lraylib -lm -lpthread -ldl -lrt -lX11
```

Repare que os **dois** arquivos `.c` são passados ao `gcc`: cada um vira um `.o`, e o *linker* junta os dois para formar o executável `atividade8`.

4 Exercícios

Exercício 1 — Nova função no módulo

Atualmente o módulo `entidade` não tem uma forma direta de verificar se uma entidade está viva.

Tarefa: adicione `bool entidadeEstaViva(Entidade *e)` ao módulo (declare em `entidade.h`, implemente em `entidade.c`) e use-a em `atividade8.c` para decidir se um inimigo deve ser removido.

Dicas:

- A função deve retornar `e->vida > 0`.
- Substitua verificações como `if (e->vida <= 0)` por `if (!entidadeEstaViva(e))`.
- Recompile com o mesmo comando de dois arquivos `.c` mostrado acima.

Exercício 2 — Reaproveitando o módulo em outro programa

Atualmente o módulo `entidade` só é usado por `atividade8.c`.

Tarefa: crie um novo arquivo `atividade8b.c` com um `main` bem simples (por exemplo, criar 3 entidades de tipos diferentes e apenas desenhá-las na tela, sem jogabilidade) que reutiliza `entidade.h/entidade.c` **sem copiar nenhuma linha de código do módulo**.

Dicas:

- Compile com `gcc atividade8b.c entidade.c -o atividade8b ...` (os mesmos flags de sempre).
- Isso demonstra o principal benefício de um módulo: o mesmo código é reaproveitado por dois programas diferentes.